

Full Length Article

Inference of hidden common driver dynamics by anisotropic self-organizing neural networks

Zsigmond Benkő ^{a,1}, Marcell Stippinger ^{a,1}, Attila Bencze ^{a,b}, Fülöp Bazsó ^a,
András Telcs ^a, Zoltán Somogyvári ^{a,c,*}

^a Theoretical Neuroscience and Complex Systems Research Group, Department of Computational Sciences, Institute for Particle and Nuclear Physics, HUN-REN Wigner Research Centre for Physics, Konkoly-Thege Miklós út 29-33, Budapest, 1121, Hungary

^b Department of Natural Sciences, Institute of Electrophysics, Kandó Kálmán Faculty of Electrical Engineering, Óbuda University, Bécsi út 96/B, Budapest, 1034, Hungary

^c Axoncord LLC., Dunakeszi u. 23, Budapest, 1048, Hungary

ARTICLE INFO

Keywords:

Latent variables
Self-organizing map
Unsupervised learning
Chaotic systems
Time series analysis
Causality analysis

ABSTRACT

We introduce the Anisotropic Self-Organizing Map (ASOM), a novel neural network-based approach for inferring hidden common drivers in nonlinear dynamical systems from observed time series. Grounded in topological theorems, our method integrates time-delay embedding, intrinsic dimension estimation, and a new anisotropic training scheme for Kohonen's self-organizing map, enabling the precise decomposition of attractor manifolds into autonomous and shared components of the dynamics. We validated ASOM through simulations involving chaotic maps, where two driven systems were influenced by a hidden nonlinear driver. The inferred time series showed a strong correlation with the actual hidden common driver, unlike the observed systems. We further compared our reconstruction performance against several established methods for identifying shared features in time series, including PCA, kernel PCA, ICA, dynamical component analysis, canonical correlation analysis, deep canonical correlation analysis, traditional self-organizing map, and recent recurrence-based approaches. Our results demonstrate ASOM's superior accuracy and robustness in recovering latent dynamics, providing a powerful tool for unsupervised learning of hidden causal structures in complex systems.

1. Introduction

1.1. The problem of detecting hidden common drivers

The detection of causal relationships between observations holds significant importance in both scientific and everyday contexts. In the realm of science, various methods have been developed to uncover directed causal connections between observed time series, drawing from disciplines such as statistics (Glymour et al., 2019; Pearl, 2000; Runge, 2018), information theory (Schreiber, 2000), and the theory of nonlinear dynamical systems (Sugihara et al., 2012) (for a recent review see Assaad et al. (2022)).

One of the most challenging problems in causal discovery is the possible existence of hidden confounders. Hidden confounders typically lead to false-positive detections of direct causal connections in the majority of causal discovery methods. For this reason, several methods have been developed to mitigate the effects of hidden confounders (Assaad et al., 2022). Such methods typically assume that the effects of a con-

founder are instantaneous between two observations or induce linear correlations between the observed data series (Chu & Glymour, 2008). Furthermore, these methods assume that there are no bidirectional connections or causal cycles between the observed variables.

Causality analysis methods that aim to differentiate between direct causal connections and the influence of a hidden common driver - without restricting the latter to an instantaneous linear effect - are based on dynamical systems theory (Benkő et al., 2018, 2024; Hirata & Aihara, 2010; Telcs et al., 2025). These methods generally assume that the observed time series is generated by deterministic, ergodic dynamical systems. In this framework, Takens' embedding theorem (Takens, 1981) shows that the attractor of a dynamical system can be reconstructed from a single scalar time series, up to a smooth, topology-preserving transformation.

Furthermore, Stark (1999) extended Takens' theorem to unidirectionally connected dynamical systems (Forced Takens Theorem). He demonstrated that the forced system can be regarded as an observation of the forcing system. Consequently, embedding the state space of

* Corresponding author.

E-mail address: somogyvari.zoltan@wigner.hu (Z. Somogyvári).

¹ These authors contributed equally.

the forced system reconstructs both the forcing (driver) and the forced (driven) dynamics, thereby capturing both cause and effect. While Takens' theorem requires the observation function to be a smooth function of the system's variables, Stark's theorem additionally requires the coupling between the two dynamical systems to possess the same smoothness.

In summary, Stark's theorem establishes that the dynamics of a cause can be reconstructed solely from observations of its effect, since the latter inherently contains the former as a component of its reconstructed attractor.

This property has been utilized by Sugihara's cross-convergent mapping (CCM) procedure, enabling the detection of unidirectional, as well as circular connections between dynamical systems (Sugihara et al., 2012). Furthermore, the CCM method has been extended to the time-frequency domain by Benkő et al. (2025). While, in theory, CCM can determine the direction of causality without assuming an observable time delay between the cause and the effect, the method has also been extended to explicitly incorporate time delays into the analysis (Ye et al., 2015). In the field of neuroscience, the initial application of CCM found that accounting for time delays remains necessary in certain cases to draw appropriate conclusions from the observations (Benkő et al., 2019).

Stark's theorem served as the inspiration for the Dimensional Causality (DC) analysis method (Benkő et al., 2018, 2024), which aims to identify and quantify the probability of all possible types of causal relationships between two time series: independence, direct or circular causal connections, and the presence of a hidden common cause. While Wójcik et al. (2001) demonstrated that the relationships between the joint and individual dimensions unequivocally determine the causal connections between the dynamical systems, Benkő et al. (2018, 2024) developed a Bayesian framework that infers these relationships based on the subadditivity of attractor dimensions, with particular focus on the joint attractor.

The effectiveness of the DC method has been validated using simulated examples of "classic" chaotic dynamical systems, including nonlinearly coupled logistic maps, coupled Lorenz systems, and Hindmarsh-Rose models. It has also been applied to EEG data, where it successfully detected the effects of visual stimulation and identified causal relationships between brain areas during epileptic seizures, accurately localizing seizure onset zones.

Notably, the DC method not only detects the presence of a hidden common driver but also estimates the dimensionality of its dynamics—an aspect that will be a key component of our work.

Our novel approach, presented here, goes beyond mere detection by enabling the reconstruction of its time series.

1.2. Inferring hidden common variables

Although the existence of a hidden confounder poses a challenge from the perspective of causality analysis, in many cases, these hidden variables that affect all observed variables are of central interest. For example, low-dimensional manifolds play a key role in population-level neural dynamics in the motor cortex (Gosztolai et al., 2024). Consequently, numerous methods have been developed to infer hidden common variables based on observed data.

The majority of these methods, such as Principal Component Analysis (PCA), Independent Component Analysis (ICA), Dynamical Component Analysis (DCA), and Canonical Correlation Analysis (CCA), assume that the observed variables are linear combinations of the hidden ones. These methods then identify these hidden linear combinations by maximizing different objective measures.

To overcome the limitations of linear models, some of these methods have been extended to nonlinear approaches. PCA and ICA have been adapted using nonlinear kernels (kPCA, kICA) (Schölkopf et al., 1997), while Deep Canonical Correlation Analysis (DCCA) (Andrew

et al., 2013) maps the observed subsystems onto one-dimensional subspaces in which the correlation between their values is maximized.

Slow Feature Analysis (SFA) also aims to reveal common features between observed time series. SFA applies a nonlinear expansion of the feature space followed by a linear projection to identify the common feature with the smoothest temporal dynamics (Wiskott, 1998; Wiskott & Sejnowski, 2002).

In contrast, assuming that the observed signals are generated by interconnected dynamical systems has led to methods where linear mixing is not presumed.

The first attempt to reconstruct the dynamics of a hidden common driver was made by Sauer (2004). Recognizing that the dynamics of a hidden common driver constitutes a shared component within the observed dynamics of the driven systems, Sauer proposed the possibility of determining this shared component through the observation of multiple systems, utilizing Takens' embedding theorem. The method described in Sauer (2010, 2004) involved decomposing one of the observed and reconstructed driven attractors into equivalence classes corresponding to fixed values in another driven attractor. This approach was demonstrated in systems where the observed attractors were at most one-dimensional manifolds, such as chaotic logistic maps driven by periodic dynamics on discrete points or periodic oscillators driven by other discrete periodic oscillators.

More recently, Gilpin (2025) advanced Sauer's method by calculating the Laplacian of the shared adjacency matrices among the driven systems and utilizing topological ordering within the equivalence classes.

Similarly to the works of Sauer (2010, 2004), Stark (1999), Stark et al. (2003), our approach is based on reconstructing the attractors of the observed systems using Takens' time-delay embedding and recognizing that information about the hidden common driver is encoded in the topological structure of the reconstructed attractors. We demonstrate that the hidden common driver can be inferred by learning a specific topological representation of the reconstructed attractor. Since self-organizing maps (SOM) are neural network models explicitly designed to learn topological representations of manifolds in an unsupervised manner (Kohonen, 1990), we employed and modified them to learn the attractor structures for hidden driver inference.

1.3. Self-organizing maps

A key tool in our work is a modified version of Kohonen's self-organizing map (Kohonen, 1982). Through an unsupervised learning process, the SOM adapts its predefined neural structure - defined by the connections between neurons - to the manifold formed by the input samples in the input space. Consequently, the network produces a topological representation, or map, of the input space.

The fundamental structure of a SOM consists of a grid of neurons. While a 2D grid is typically used, 1D or higher-dimensional grids can also be employed. Each neuron in the grid is characterized by a normalized synaptic weight vector that connects the input space to the neuron, effectively marking the center of its receptive field in the input space.

During the learning process, the input samples are presented sequentially. The activation of a neuron at each presentation depends on the distance between the given sample and the neuron's receptive field center, i.e., its weight vector. A winner-takes-all mechanism is introduced, fostering competition among activated neurons. The neuron whose receptive field center is closest to the given sample wins the competition and drives the actual learning step. The synaptic weights of all neurons adjust slightly toward the given sample, but the learning rate varies based on each neuron's distance from the winner along the inner neural grid. As a result, the synaptic weight of the winner is modified the most significantly, while the weight modification decreases as the distance from the winner increases.

This competition among neurons for input samples leads to an even distribution of receptive field centers along a homogeneous input manifold. When dealing with inhomogeneous samples, the density of

receptive field centers becomes proportional to the sample density of the input space. Moreover, neurons that are located close to each other in the inner grid also tend to have receptive fields that occupy nearby positions in the input space. As a result, the grid forms a smooth mapping of the input manifold.

In our framework, the Anisotropic Self-Organizing Map (ASOM) is trained using not just one but two sets of inputs derived from the embedding of the two driven systems. The original training method has been modified to enable anisotropic map fitting. By incorporating this modified anisotropic map-fitting, we can effectively decompose the observed attractor into submanifolds that describe the autonomous dynamics of the driven system, as well as dimensions that correspond to the dynamics of the hidden common driver.

2. Methods

2.1. Simulations

Two types of discrete-time, continuous-space, one-dimensional dynamical systems were simulated to generate time series for analysis: logistic maps and tent maps.

2.1.1. Logistic maps

The basic form of the logistic map is:

$$x(t+1) = rx(t)(1-x(t)). \quad (1)$$

Depending on the value of the parameter r , the system exhibits periodic or chaotic behavior or converges to a fixed point. The exact values of the parameters used in our simulations are given in the [Appendix A, Table A.2](#).

To demonstrate the topological relationships between driver and driven dynamical systems, we simulated unidirectionally coupled logistic maps:

$$\begin{aligned} x(t+1) &= r_1 x(t)(1-x(t)), \\ y(t+1) &= r_1 y(t)(1-y(t)-\beta_1 x(t)). \end{aligned} \quad (2)$$

This form of coupling implements a nonlinear connection between the two systems, where the strength of the effect is determined by the β_1 parameter.

For the case of circular nonlinear coupling, our example dynamical system is defined as follows:

$$\begin{aligned} x(t+1) &= r_2 x(t)(1-x(t)-\beta_2 y(t)), \\ y(t+1) &= r_2 y(t)(1-y(t)-\beta_3 x(t)), \end{aligned} \quad (3)$$

The ASOM training process was demonstrated on a triad of coupled logistic systems:

$$\begin{aligned} z(t+1) &= r_3 z(t)(1-z(t)) + \eta_1(t), \\ x(t+1) &= r_3 x(t)(1-x(t)-\beta_4 z(t)) + \eta_2(t), \\ y(t+1) &= r_3 y(t)(1-y(t)-\beta_5 z(t)) + \eta_3(t), \end{aligned} \quad (4)$$

where η_{1-2-3} represents additive Gaussian white noise terms.

2.1.2. Tent maps

The basic form of the update rule for the tent map is given by the following equation:

$$x(t+1) = 1 - 2|x(t) - 1/2|. \quad (5)$$

However, for our simulations, we used a modified, tilted map:

$$\begin{aligned} x(t+1) &= T_\alpha(x(t)) = \frac{1}{2\alpha(\alpha-1)}|x(t)-\alpha| \\ &\quad + \left(\frac{1}{\alpha} + \frac{1}{2\alpha(\alpha-1)}\right)(x(t)-\alpha) + 1. \end{aligned} \quad (6)$$

The peak value of this triangular map is 1, but its position is now at α .

A coupled triad of tent maps was simulated for systematic performance evaluation:

$$\begin{aligned} z(t+1) &= T_{\alpha_1}(z(t)), \\ x(t+1) &= T_{\alpha_2}(x(t) - \beta_6 z(t)), \\ y(t+1) &= T_{\alpha_3}(y(t) - \beta_7 z(t)). \end{aligned} \quad (7)$$

The α parameters were chosen uniformly from the interval $[0.1, 0.5]$, and the β connection strength parameters were selected from the interval $[0.1, 0.5]$.

2.2. Embedding and cross-mapping

The topological relationships between the observed systems are demonstrated on the reconstructed attractor manifolds using time-delay embedding, following Takens' embedding theorem:

$$\begin{aligned} X(t) &= [x(t), x(t+\tau), \dots, x(t+(m-1)\tau)], \\ Y(t) &= [y(t), y(t+\tau), \dots, y(t+(m-1)\tau)]. \end{aligned} \quad (8)$$

In the specific examples, we set $\tau = 1$ time-step and $m = 3$, embedding all time series into three dimensions. To distinguish between scalar-valued and vector-valued time series, lowercase letters are used for the former, while capital letters denote the embedded vector-valued time series.

Although Convergent Cross-Mapping (CCM) ([Sugihara et al., 2012](#)) is unable to distinguish the effect of a hidden common driver from direct causality, it was utilized here to demonstrate the topological relationships between manifolds. Thus, cross-mapping from system X to system Y is performed similarly to [Sugihara et al. \(2012\)](#):

- Initially, a random state $X(t_s)$ is selected from system X .
- Next, the K nearest neighbor (KNN) states $X(t_k)$, $k \in \{1, \dots, K\}$ are identified in the reconstructed state space of system X using Euclidean distances.
- Finally, the corresponding states $Y(t_k)$, $k \in \{1, \dots, K\}$ from system Y are retrieved.

2.3. Dimensional causality analysis

The initial step of our analysis pipeline is to perform the Dimensional Causality analysis ([Benkő et al., 2018, 2024](#)) on the two observed time series. This analysis assigns probabilities to the five basic possible causal relationships between the two time series, including the existence of a hidden common driver between otherwise independent systems. If Dimensional Causality assigns a high probability to the presence of a hidden common driver, then the analysis method based on the ASOM network presented here can be used to infer that hidden common driver.

2.4. Dimensions of the ASOM

Proper construction of the Anisotropic Self-Organizing Map (ASOM) requires knowledge of the dimensions of both the observed and hidden common driver dynamics. Within the Dimensional Causality framework, dimension estimates of the embedded manifolds are calculated using a K -nearest neighbor (KNN)-based method proposed by [Farahmand et al. \(2007\)](#) and modified by [Benkő et al. \(2020\)](#).

To estimate the intrinsic dimension of the embedded time series $X(t)$, we iterate through all points and search for their K^{th} and $2K^{\text{th}}$ nearest neighbors. Let the distance of the K^{th} neighbor from the original point $X(t)$ be denoted as $R_K(t)$. Then, the local dimension $d_K(X(t))$ around $X(t)$ is estimated as follows:

$$d_K(X(t)) = \frac{\ln(2)}{\ln(R_{2K}(t)/R_K(t))}. \quad (9)$$

While [Farahmand et al. \(2007\)](#) used the mean of the local dimension estimators as a global estimate, [Benkő et al. \(2020\)](#) showed that the

median of the local dimensions provides a more robust and unbiased global estimate of the global manifold dimension (D_X):

$$D_X = \text{Median}(d_K(X(t))). \quad (10)$$

The intrinsic dimension D_Y of the time series Y is estimated in a similar way.

To determine the optimal neighborhood parameter K , two effects must be taken into account: Higher K values add robustness to dimension estimation against noise, but introduce more bias if the manifolds are highly curved. In our case, the dimension estimations were averaged over the interval $K \in \{10, 11, \dots, 20\}$, where the dimension estimations were relatively stable. This range worked well given the actual noise level and the high curvature of the manifolds.

The dimension of the hidden common driver, denoted as D_Z , can be estimated by calculating the Mutual Information Dimension between the time series $x(t)$ and $y(t)$ (Sugiyama & Borgwardt, 2013). This involves the dimensions of the two observed systems and the dimension of their joint system, denoted as D_J :

$$D_Z = D_X + D_Y - D_J, \quad (11)$$

where the joint manifold J is the direct product of the X and Y manifolds, residing in an embedding space of double dimension:

$$J(t) = [X(t), Y(t)]. \quad (12)$$

Dynamical systems, especially chaotic ones, often produce fractal attractors, resulting in non-integer dimensions. As the dimension of the ASOM is an integer, the closest integer value should be used to define the dimension of the ASOM.

In our demonstration example, the hidden common cause, as well as the driven consequences, are described by logistic dynamics. As a result, the hidden common driver exhibits dynamics with a dimension close to one, while both driven systems display dynamics with dimensions close to 2. Thus, a 2D anisotropic grid is necessary to fit the observed dynamics, with one of these two dimensions representing the dynamics of the hidden common driver.

The anisotropic SOM is constructed using a 2D grid of $N_1 \times N_2$ nodes. The shape of the grid should conform to the shape of the manifold it aims to fit. While it is not straightforward to measure the exact linear lengths of a curved manifold, our simulations resulted in elongated manifolds. Therefore, unless otherwise stated, we used $N_1 = 40$ and $N_2 = 20$.

Each node is characterized by its position in the 2D grid, described by indices i and j , and the center of its receptive field in the reconstructed Y space, represented by $C_{i,j}$ vectors. The dimension of the $C_{i,j}$ vectors should be equal to the dimension of the embedding space that is 3 in the presented cases. The first objective of the training procedure is to evenly distribute the $C_{i,j}$ points across the manifold Y , ensuring that the 2D grid forms a discretized topographical map of the 2D manifold Y .

Additionally, anisotropic fitting is required so that the first dimension ($N_1 = 40$ nodes long) follows the submanifolds corresponding to the autonomous dynamics of system Y , while the second dimension ($N_2 = 20$ nodes long) corresponds to the different states of the hidden common cause. Each autonomous dynamics submanifold consists of points among which the system Y moves while the value of the hidden common cause Z remains fixed. Thus, movement along these submanifolds is governed by the autonomous dynamics, whereas transitions between submanifolds are driven by the dynamics of the hidden common cause.

In general, the ASOM should have D_Y dimensions to effectively learn the Y manifold. Among these dimensions, D_Z describes the dynamics of the common driver, while the remaining $D_Y - D_Z = D_J - D_X$ dimensions represent the autonomous dynamics of the system Y .

2.5. Anisotropic training of the ASOM network

Simulated time series of different lengths from $T = 100$ to $T = 20000$ time steps were used to train the ASOM network. Two different initial

distributions of the $C_{i,j}$ ($s = 0$) vectors were used and tested. In the first, the vectors $C_{i,j}$ ($s = 0$) were randomly distributed with a uniform distribution within the unit cube. In the second version, the $C_{i,j}$ ($s = 0$) vectors were placed in a regular grid on a unit square at a height (third coordinate) of 1, forming a “falling veil” during the initial phase of learning. The parameters of the ASOM trainings are summarized in Appendix A, Table A.3.

The anisotropic training consisted of two nested loops: The outer loop typically ran $N = 20000$ times. In each iteration, a random state $X(t_s)$ from the X manifold was chosen and mapped to the corresponding $Y(t_s)$ point. The activation of the nodes depended on the distance between the presented $Y(t_s)$ sample and their receptive field centers $C_{i,j}(\gamma)$. The overall winner node $C_{i^*,j^*}(\gamma)$ of the ASOM, whose center was closest to $Y(t_s)$, was then selected:

$$i^*, j^* = \arg \min_{ij} (||C_{i,j}(\gamma) - Y(t_s)||). \quad (13)$$

In the next step, the nearest neighbors $X(t_k)$, $k \in \{1, \dots, K\}$ ($K = 10$) were searched for and mapped to the corresponding $Y(t_k)$, $k \in \{1, \dots, K\}$ elements. These $Y(t_k)$ points formed a one-dimensional submanifold in the Y space.

The second loop starts here, where the $Y(t_k)$, $k \in \{1, \dots, K\}$ elements are presented to the SOM one after another. At this stage, the training procedure becomes anisotropic: The distance between the current $Y(t_k)$ and only the $C_{i^*,j^*}(\gamma)$ row of $C_{i^*,j^*}(\gamma)$ is calculated, and the actual winner $C_{i^a,j^a}(\gamma)$ is selected based on the smallest distance:

$$i^a = \arg \min_i (||C_{i,j^*}(\gamma) - Y(t_k)||). \quad (14)$$

The learning weights are then computed for all nodes according to their anisotropic distances from $C_{i^a,j^a}(\gamma)$:

$$w_{i,j}(\gamma) = e^{-((i-i^a)^2/\sigma_1^2 + (j-j^a)^2/\sigma_2^2)}. \quad (15)$$

Both $\sigma_1(s)$ and $\sigma_2(s)$ depend on the number of simulation steps completed, resulting in an anisotropic shrinkage of the learning area within the grid during training:

$$\begin{aligned} \sigma_1(s) &= 10 \cdot e^{-s/N}, \\ \sigma_2(s) &= 20 \cdot e^{-(s \ln 5)/N}. \end{aligned} \quad (16)$$

As the final step within the second loop, the centers of perception vectors for all grid nodes are updated:

$$C_{i,j}(\gamma + 1) = C_{i,j}(\gamma) + \epsilon(s) \cdot w_{i,j}(\gamma) \cdot (Y(t_k) - C_{i,j}(\gamma)), \quad (17)$$

where the learning rate $\epsilon(s)$ decreases with the number of simulation steps in the outer loop, leading to the stabilization of the centers by the end of the learning process:

$$\epsilon(s) = 0.2 \cdot e^{-(s \ln 20)/N}. \quad (18)$$

After all $Y(t_k)$, $k \in \{1, \dots, K\}$ points have been presented to the ASOM, the second loop terminates, and a new round of the first loop begins by randomly selecting a new $X(t_s)$. For a pseudocode description, see Algorithm 1.

2.6. Readout

To extract the estimated values of the hidden common driver from the well-trained ASOM, we examine the second index of the winner node. For each time instance t in the test set, the embedded input patterns $X(t)$ and $Y(t)$ are presented to the ASOM, and the winner node is selected similarly to the training phase, according to Eq. 13. The second index $j^*(t)$ of the winner node provides a discretized estimate $\hat{z}(t)$, representing a topologically smooth mapping of the hidden common driver.

Although the real numerical values of the hidden common driver cannot be directly obtained, the time series of the $j^*(t)$ winner indices can be transformed into comparable values by applying a smooth transformation. Both time series are mean-corrected to zero and normalized by their standard deviations, assuming that scaling and shifting, as

Algorithm 1 Training algorithm for ASOM.

```

 $X(t) \leftarrow [x(t), x(t + \tau), \dots, x(t + (m - 1)\tau)]$  ▷ Time delay embedding of time series
 $Y(t) \leftarrow [y(t), y(t + \tau), \dots, y(t + (m - 1)\tau)]$ 
 $\gamma \leftarrow 0$ 
 $C_{i,j}(\gamma) \leftarrow r_{ij}$  ▷ Initialize the grid
for  $s = 0$  to  $N$  do
  Compute  $\sigma_1(s), \sigma_2(s), \epsilon(s)$  ▷ Set learning rates
   $t_s \leftarrow r_s$  ▷ Pick a random time index
   $i^*, j^* \leftarrow \arg \min_{i,j} (||C_{i,j}(\gamma) - Y(t_s)||)$  ▷ Find the nearest element on the grid
   $t_1, \dots, t_K \leftarrow \text{KNN}(X(t_s))$  ▷ Find the time index of nearest neighbors
  for  $k = 1$  to  $K$  do
     $i^a \leftarrow \arg \min_i ||C_{i,j^*}(\gamma) - Y(t_k)||$  ▷ Find the nearest grid-point within the selected row
     $w_{i,j}(\gamma + 1) \leftarrow \exp[-((i - i^a)^2 / \sigma_1^2(\gamma) + (j - j^*)^2 / \sigma_2^2(\gamma))]$  ▷ Compute weights
     $C_{i,j}(\gamma + 1) \leftarrow C_{i,j}(\gamma) + \epsilon(s)w_{i,j}(\gamma)(Y(t_k) - C_{i,j}(\gamma))$  ▷ Update grid-point coordinates
     $\gamma \leftarrow \gamma + 1$ 
  end for
end for

```

smooth transformations, are sufficient for comparison. To assess scale-independent similarity, we calculate the Pearson correlation between $z(t)$ and $\hat{z}(t)$.

The performance of our ASOM method was compared with several previously proposed methods for extracting common components from multivariate time series. These alternative methods are described in [Appendix B](#).

2.7. Calculation of fitting error

The results of the training process are stored in the $C_{i,j}$ network structure. The fitting error is calculated to assess the quality of this structure *without knowledge of the ground truth data*. It measures the mean squared error of the manifold learning by computing the difference between the center coordinates of the actual winner node and the actual sample:

$$E = \frac{1}{T} \sum_{t=1}^T ||Y(t) - C_{i^*,j^*}(t)||^2. \quad (19)$$

This fitting error naturally decreases with increasing grid size due to higher resolution. Thus, to enable the comparison of fitting errors across different grid sizes, we introduce a normalized fitting error:

$$E_N = E \sqrt{N_1 + N_2^2}, \quad (20)$$

where N_1 denotes the number of nodes along the autonomous dynamics dimension, and N_2 represents the number of nodes along the hidden driver dynamics dimension. The intuition behind the different exponents is that the hidden common driver reconstruction error depends more on the resolution along its corresponding dimension (N_2) than on the size of the transversal dimension (N_1).

Comparing fitting errors allows for optimizing parameter selection and choosing between the results of two training processes run with the same parameters.

2.8. Workflow

In summary, our workflow involves the following steps ([Fig. 1](#)):

1. Time-delay embed the two observed signals ($x(t)$ and $y(t)$), resulting in two manifolds created by the vector-valued time series $X(t)$ and $Y(t)$.
2. Construct the joint state space as a direct product $J(t) = [X(t), Y(t)]$.
3. Estimate the dimensions D_X , D_Y , and D_J of the manifolds X , Y , and J , respectively.
4. Calculate the mutual dimension: $D_Z = D_X + D_Y - D_J$.
5. Create the anisotropic neural grid of the ASOM, with $D_Y - D_Z$ dimensions for the autonomous dynamics of system Y and D_Z dimensions for the shared dynamics.

6. Perform anisotropic training of the ASOM on $Y(t)$ using the $X(t)$ time series.
7. Present the test set $Y(t)$ and extract $\hat{z}(t)$ from the trained ASOM.
8. Repeat the training process and compare the resulting fitting errors to optimize the network structure and parameter setup.

Our implementation can be found at <https://github.com/tncslab/aniSOM>.

3. Results

In the first part of the Results section, we examine the topological relationships between the attractors of coupled dynamical systems. The insights gained from applying cross-mappings between attractors are generalized using fundamental topological theorems, which form the theoretical foundation of the ASOM method. We then present an example of Dimensional Causality analysis on a simulated system to demonstrate the applicability of ASOM and to determine appropriate network dimensions. This is followed by simulation examples illustrating the temporal evolution of the ASOM network during training, the readout process, and the evaluation of the hidden common driver time series using well-trained networks. Next, we introduce a fitting error measure to assess the quality of network training in the absence of a ground-truth signal, enabling the optimization of both network parameters and the training process. The robustness of ASOM to noise and the effects of varying network size are evaluated through multiple simulations. Finally, we compare the precision and efficiency of ASOM with several established methods for extracting common components from multivariate time series of various lengths.

3.1. Cross-mapping between coupled dynamical systems

The topological relations between attractor manifolds of coupled dynamical systems are examined by cross-mapping on two examples: first, between bidirectionally coupled logistic maps, and second, between unidirectionally coupled logistic maps (see [Fig. 2](#)). In the bidirectionally coupled case ([Eq. 3](#)), the two reconstructed manifolds are topologically equivalent, their dimensions are equal (two in this case), and cross-mapping functions effectively in both directions ([Fig. 2 A–D](#)). Mapping a small neighborhood around any point on one manifold results in a comparably small patch of neighboring points on the other manifold.

Next, we examined the properties of cross-mapping in unidirectionally coupled systems ([Eq. 2](#)). Here, the dimensions of the two manifolds differ: the cause $X(t)$ forms a 1D manifold in the embedding space, while the consequence $Y(t)$ forms a 2D manifold in the 3D embedding space ([Fig. 2 E–H](#)). Due to this dimensional difference, the two manifolds can-

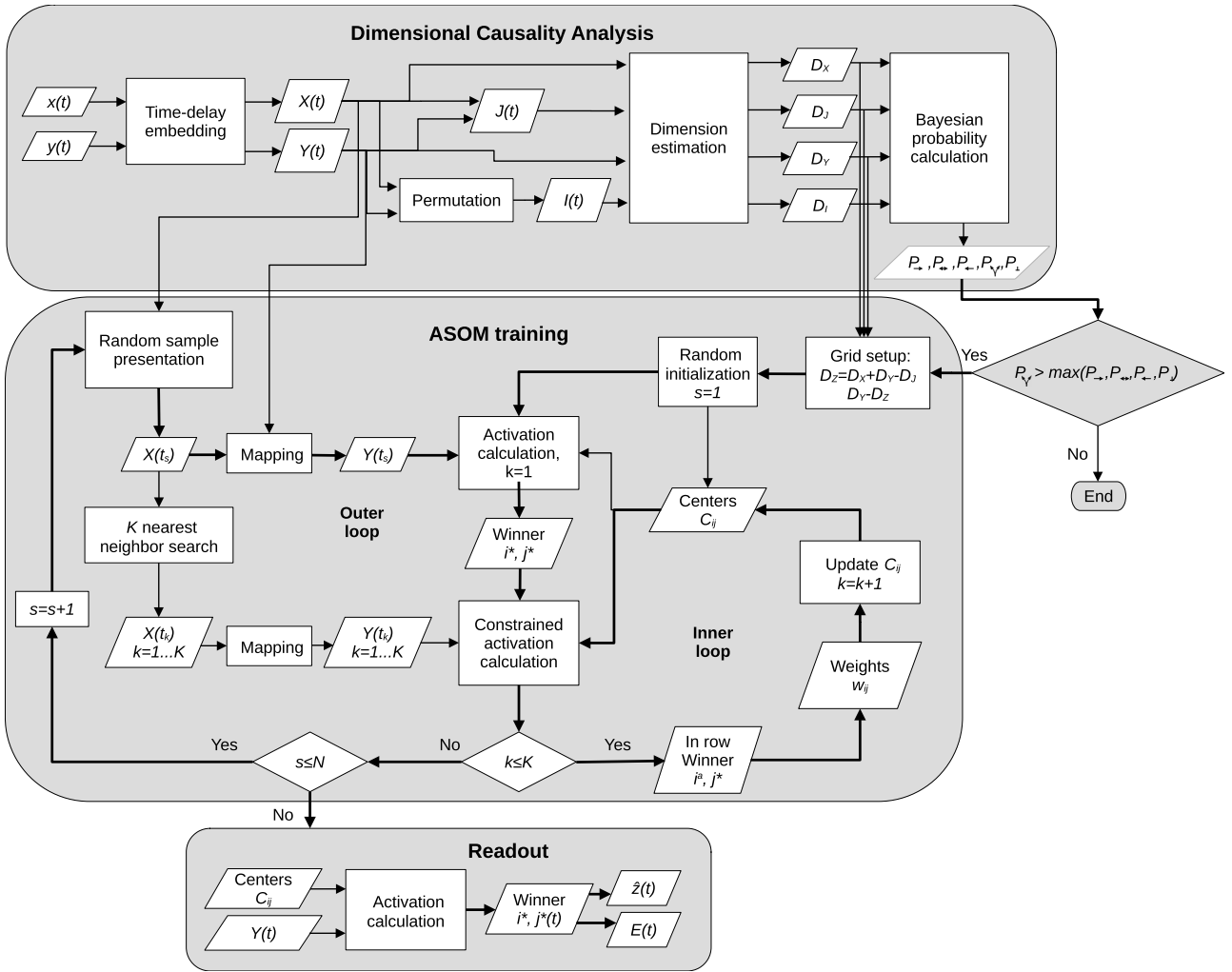


Figure 1. Workflow chart of our ASOM method for inferring hidden common driver time series. The main input consists of two scalar-valued time series, denoted by $x(t)$ and $y(t)$, while the corresponding embedded vector-valued time series are denoted by $X(t)$ and $Y(t)$. The joint and independent (time permuted) joint time series are represented by $J(t)$ and $I(t)$, respectively. The outputs of the procedure are the estimated hidden common driver time series $\hat{z}(t)$, and the fitting error $E(t)$. The primary control flow within the inner and outer loops is emphasized by thick lines, while thin lines denote the data flow.

not be topologically equivalent: cross-mapping implements a surjective but not an injective function from Y to X .

While cross-mapping a small neighborhood from Y (Fig. 2 H) to X results in a similarly small patch in X (Fig. 2 G), cross-mapping a small neighborhood in the X manifold (Fig. 2 E) forms a 1D submanifold within the 2D manifold of Y (Fig. 2 F). As system Y moves along these submanifolds at a given value of the cause X , these submanifolds describe a component of Y 's dynamics, which we refer to as autonomous dynamics. Meanwhile, the cause X drives system Y between these submanifolds, meaning each submanifold represents a specific value of the cause X .

Now, consider the case of a hidden common cause (Eq. 4), where two observed systems X (Fig. 3(A)) and Y (Fig. 3(B)) are both driven by the variable Z of a third, unobserved logistic map, (Fig. 3(C)). Both observed systems form two-dimensional manifolds, similar to the circularly coupled case, but the behavior of cross-mapping differs.

Cross-mapping small patches from manifold X (orange and green patches in Fig. 3(A)) results in small patches in the hidden cause Z (Fig. 3(C)), whereas the same patches are mapped onto two distinct 1D submanifolds within manifold Y (Fig. 3(B)). Thus, similar to the unidirectional case, the 2D manifold Y can be decomposed into 1D submanifolds, however, in this case, each submanifold corresponds to a specific value of the hidden common cause Z . The autonomous dynamics moves

system Y along these submanifolds, while the dynamics of the hidden common cause drive system Y between these submanifolds.

Here, we have demonstrated an example where the Y manifold is decomposed into a bundle of 1D submanifolds parameterized by the values of the hidden common driver. However, an important question arises: how general is this property, and what necessary conditions must the system satisfy to allow such a decomposition?

3.2. Theoretical foundations of the ASOM method

The preimage theorem, also known as the regular level set theorem in differential topology (Banyaga, 2004), states that if $f : Y \rightarrow Z$ is a smooth map and $Z_i \in Z$ is a regular value of f , then the preimage $f^{-1}(Z_i)$ is a submanifold of Y with dimension $D_Y - D_Z$ and the codimension of $f^{-1}(Z_i)$ is equal to the dimension D_Z of Z .

This is exactly what we want: the preimage of Z_i forms an equivalence class where $Y_1 \sim Y_2$ if and only if $f(Y_1) = f(Y_2) = Z_i$. These equivalence classes for all $Z_i \in Z$ form a fiber bundle. The manifold Y is decomposed into these fibers, and the complementary dimension of the fibers corresponds to the dimension of the Z manifold.

Thus, according to the preimage theorem, two necessary conditions must be satisfied to allow this decomposition: the map $f : Y \rightarrow Z$ must be smooth, and the points $Z_i \in Z$ must be regular. A point $Z_i \in Z$ is

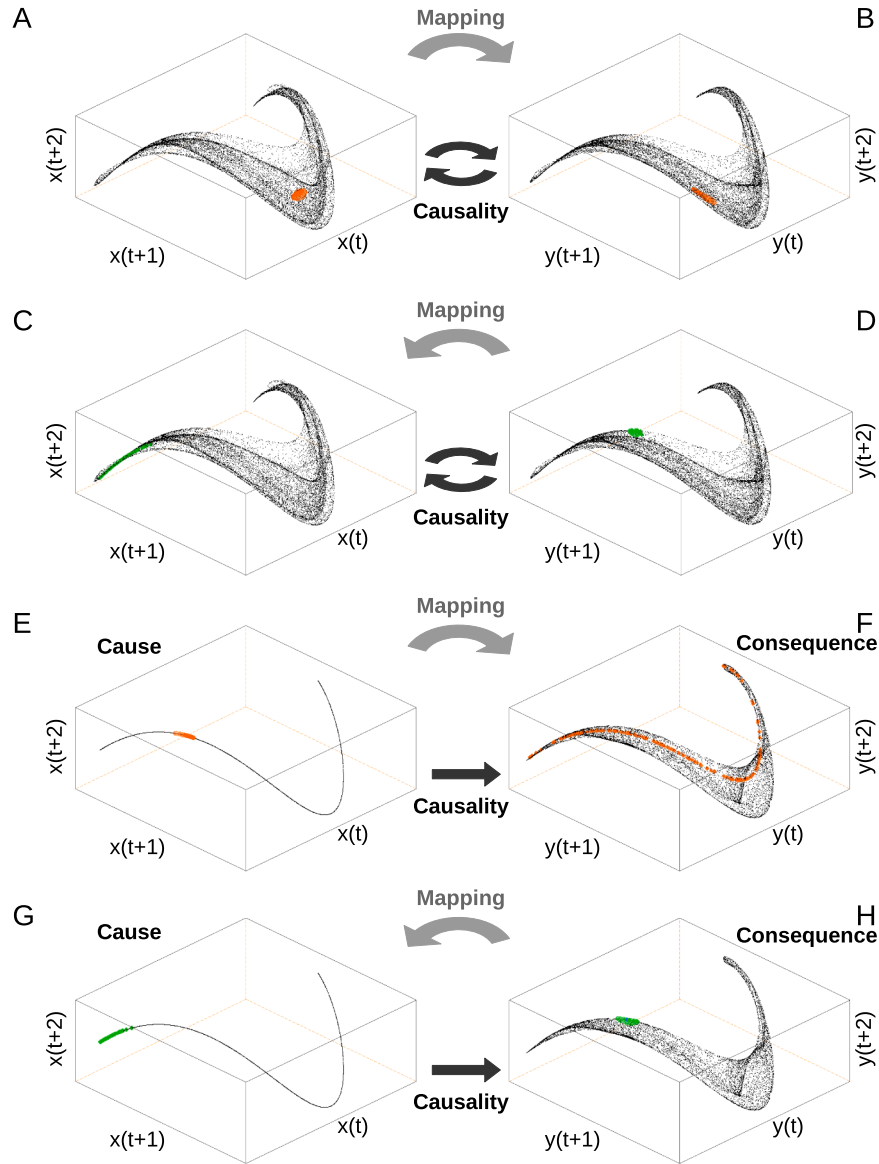


Figure 2. Cross-mapping between coupled logistic systems, represented in 3D embeddings. A–D: In the case of circular coupling between the systems, the mapping behaves similarly in both directions. A local neighborhood in system X (A) is mapped onto a local neighborhood in the state space of system Y (B). Similarly, a local neighborhood in the state space of Y (D) is mapped onto a local neighborhood in system X (C). E–H: However, in unidirectional coupling, the dimensions of the two systems differ, leading to asymmetry in the projection behavior. A small local neighborhood in the cause (E) is mapped onto a one-dimensional submanifold in the state space of the consequence (F), while a small local neighborhood in the consequence (H) is mapped onto a small local neighborhood in the cause (G).

considered a regular value of f if, for all $Y_j \in f^{-1}(Z_i)$, the differential map $df_{Y_j} : T_{Y_j}Y \rightarrow T_{Z_i}Z$ is surjective. Here, $T_{Y_j}Y$ and $T_{Z_i}Z$ denote the tangent spaces of Y and Z at the points Y_j and Z_i , respectively. In other words, this condition requires that $\text{rank}(df_{Y_j}) = D_Z$ for all Y_j in the preimage. If a point Y_j is not regular, it is called a critical or singular point.

Sard's theorem (Sard, 1942) states that for any smooth function on a smooth manifold, regular points are generic: almost all points in a manifold are regular, while critical points are isolated and have measure zero. Since regular values form an open and dense set, it is always possible to choose a regular point in an arbitrarily small neighborhood of a critical point. Thus, this condition does not limit the applicability of the theorem in our case.

Consequently, we conclude that the decomposition is viable (at almost every point) if $f : Y \rightarrow Z$ is smooth.

3.3. Dimensional causality analysis

Before the inference of the hidden common driver began, the first step was to perform Dimensional Causality (DC) analysis (Benkő et al., 2018, 2024) to investigate the existence of the hidden common driver and measure the dimensions of the systems (Fig. 4). The steps of our analysis pipeline are demonstrated on a simulated time series of a triad of coupled logistic maps, similar to the example presented earlier: the common driver (Fig. 4(A), green, $z(t)$) drives the two observed systems (Fig. 4(B), red, $x(t)$, and Fig. 4(C), blue, $y(t)$).

For the DC analysis, a segment of 5000 time steps from the observed time series was used and embedded into 4D spaces of which 3D is shown in (Fig. 4(C) and G). Fig. 4(E) shows the estimated dimensions of all manifolds. The dimensions of both X and Y manifolds were close to two:

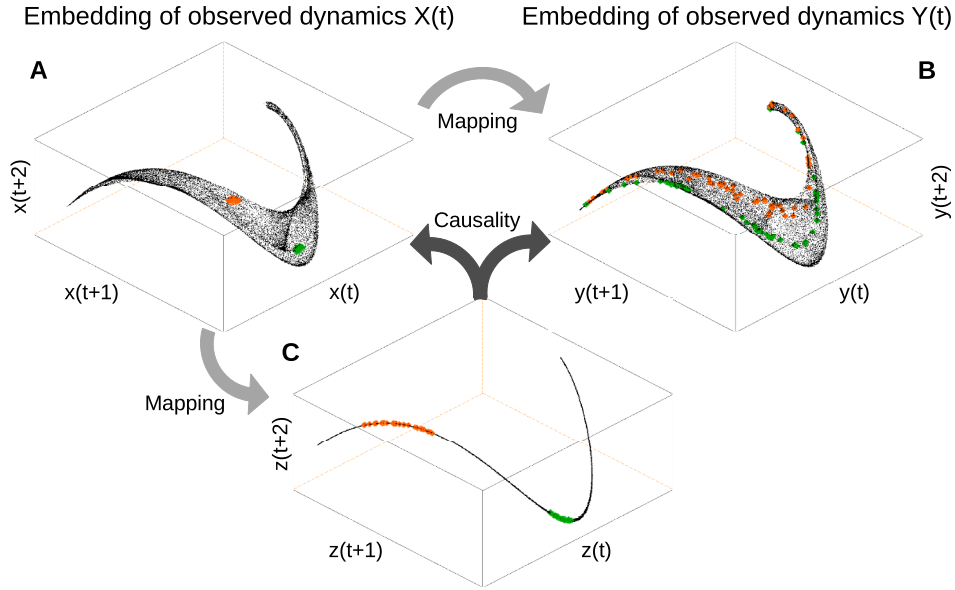


Figure 3. Cross-mapping with a hidden common driver. The Z system serves as the hidden common driver (C), influencing the two observed uncoupled systems X and Y , shown in (A) and (B), respectively. Cross-mapping the orange and green local patches, which form two local neighborhoods in the state space of X (A), results in two well-localized patches in the state space of the unobserved Z system (C). However, mapping these local neighborhoods onto the Y system produces two one-dimensional submanifolds (B).

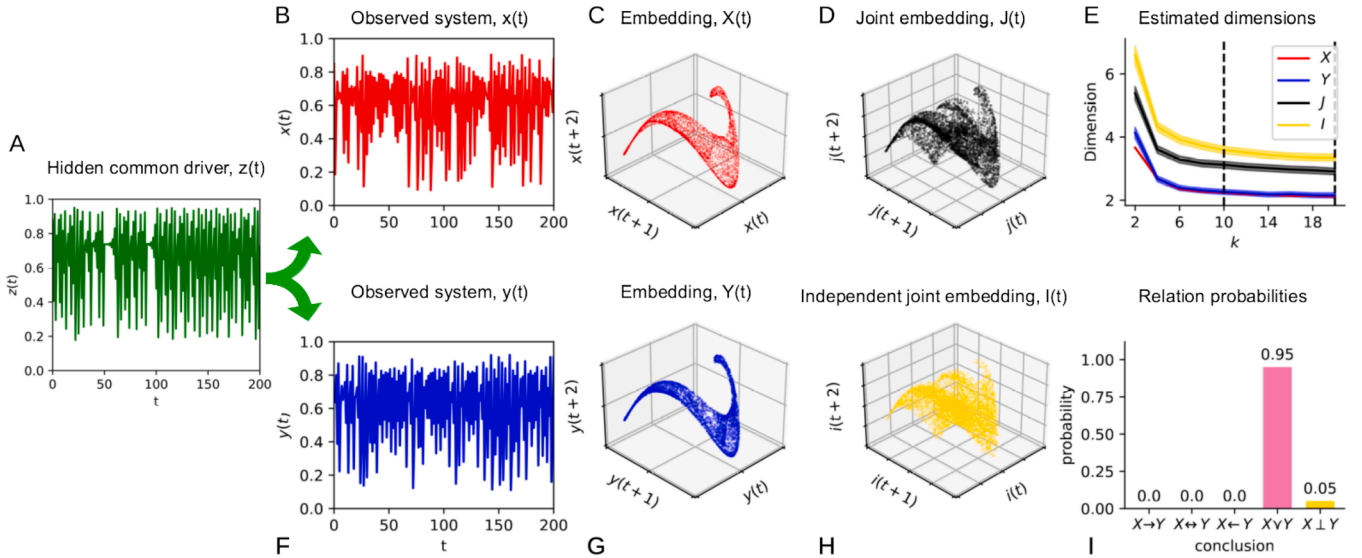


Figure 4. Dimensional Causality analysis. A system of three connected logistic maps was simulated and analyzed. A: time series of the hidden common driver, $z(t)$, green. B and F: time series of both observed time series ($x(t)$, red, and $y(t)$, blue). C and G: The observed time series are embedded in 3 dimensions. D: Joint embedding ($J(t)$, black). H: Time-permuted independent joint embedding ($I(t)$, yellow). E: Estimated dimensions as a function of the neighborhood size (k). The interval between 10 and 20 neighbors was used for Bayesian inference. I: Estimated posterior probabilities of the five basic causal relations: $X \rightarrow Y$: X drives Y ; $X \leftrightarrow Y$: bidirectional coupling; $X \leftarrow Y$: Y drives X ; $X \vee Y$: a hidden common cause drives both; $X \perp Y$: X and Y are independent. The dimensional analysis assigns the highest probability to the existence of a hidden common driver.

$$D_X = 2.17 \pm 0.05, \quad D_Y = 2.19 \pm 0.04 \quad (\text{mean} \pm \text{SD})$$

while the estimated joint dimension of the systems was

$$D_J = 3 \pm 0.07,$$

and the dimension of the independently time-permuted joint system was

$$D_I = 3.45 \pm 0.13.$$

According to the principle of DC analysis, the relationships among D_X , D_Y , D_J , and D_I unequivocally determine the causal structure of

the observed systems. Specifically, in this case:

$$\max(D_X, D_Y) < D_J < D_I. \quad (21)$$

This result implies that the X and Y systems are unconnected, but a hidden common driver exists. The Bayesian evaluation procedure for the dimensional relations assigns a high probability to the existence of the hidden common driver (Fig. 4(F)).

For further details on the DC analysis, the reader is referred to Benkő et al. (2024).

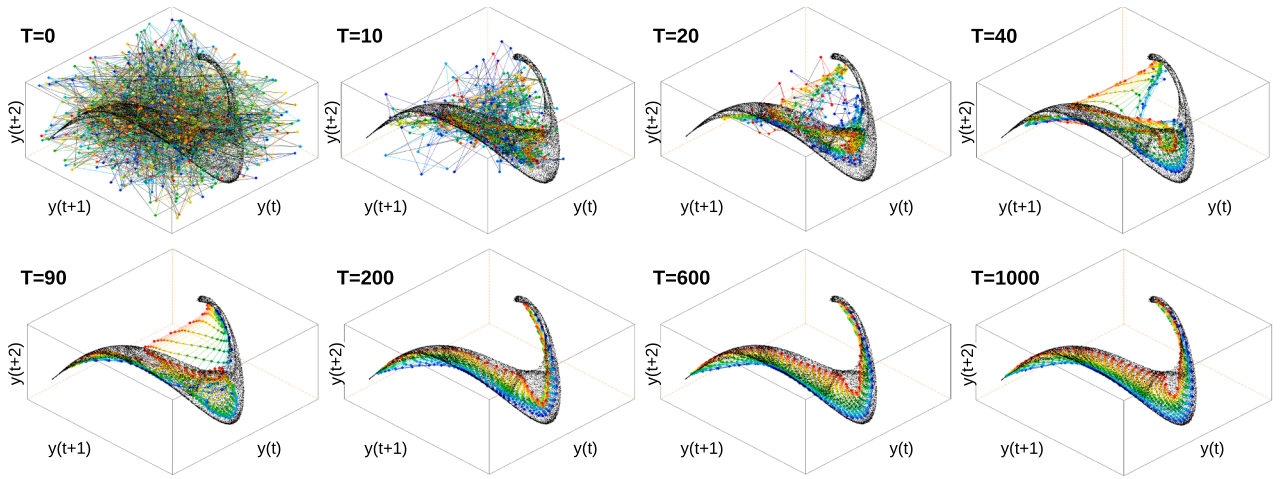


Figure 5. Development of the ASOM during anisotropic learning. At each time-step (s), a random point in manifold X was selected, and its local neighborhood of $K = 20$ points was presented to the network, forming a fiber in manifold Y . The anisotropic learning procedure fits rows of the network (marked by different colors) to these fibers. The 20×40 nodes of the grid represent the $C_{i,j}$ centers of the receptive fields of the neurons, while the edges correspond to the predefined neighborhood structure of the network.

Based on the above results, the dimension of the hidden common driver can be estimated using Eq. 11:

$$D_Z = D_X + D_Y - D_J = 1.36. \quad (22)$$

The autonomous dynamics dimension of X is given by

$$D_X - D_Z = 0.81, \quad (23)$$

while the autonomous dynamics dimension of Y is

$$D_Y - D_Z = 0.83. \quad (24)$$

These results align with the properties of the simulated systems and serve as a basis for determining the dimensions of the ASOM network. Since the ASOM network requires integer dimensions, both the autonomous dynamics and the hidden common driver dynamics are rounded to 1. Thus, a $1 + 1$ -dimensional ASOM is set up to properly learn the attractor manifold structure of each observed system.

3.4. Training of the ASOM network

The task of the anisotropic training of the ASOM network is to learn the decomposition of the attractor manifold, as revealed in the Dimensional Causality analysis.

Fig. 5 illustrates an example of the development of the ASOM during learning. At the beginning of the training process, the $C_{i,j}$ centers of the receptive fields of the neurons are uniformly distributed within the unit cube. At each time step, a random point from the manifold $X(t)$ is selected, and its local neighborhood of $K = 20$ points is presented to the network, forming a fiber in the manifold Y . As learning progresses, the predefined 40×20 grid structure of the network gradually aligns with the embedded manifold of $Y(t)$, fitting more smoothly over time.

Fig. 6(A) shows a well-trained ASOM that successfully captures the topology of the manifold. The first dimension of the 2D map, represented by colored rows, aligns along the submanifolds describing the autonomous dynamics, while the second dimension, marked by different colors, spans across all submanifolds and corresponds to the values of the hidden common driver Z . Fig. 6(B) presents the manifold Z color-coded according to the winner row index j^* of the ASOM corresponding to the colored rows in Fig. 6(A).

After completing the training, the $C_{i,j}$ weights of the network were frozen, and a new data series of length $N = 10000$ steps, generated using the same parameters but not included in the training process, was

used to evaluate the precision of the reconstruction. The discretized estimates of the hidden common driver were read out from the network as the second index j^* of the winner node. The reconstructed data was then normalized by setting its median to zero and its standard deviation to one (Fig. 6(C), green line) and compared to the ground truth values of the hidden common driver, also normalized similarly (Fig. 6(C), black line). Remarkably, the reconstructed hidden common driver closely follows the chaotic dynamics of the actual driver.

The correlation between the inferred $\hat{z}(t)$ and the ground truth $z(t)$ is presented (Fig. 6(D)) and quantified by the absolute value of the Pearson correlation coefficient. In this specific example, the absolute correlation between the inferred and ground truth hidden driver reached 0.91. In contrast, the observed variables $x(t)$ and $y(t)$ exhibited only weak instantaneous absolute correlations of 0.24 and 0.2 with the hidden $z(t)$, respectively.

Interestingly, despite the time delay between the hidden common driver and the driven systems being only one time step, the maximum absolute correlation coefficients were reached at different delays: the peak absolute value of the cross-correlation function between $x(t)$ and $z(t)$ was 0.42, occurring with a two-timestep delay, while the peak correlation between $y(t)$ and $z(t)$ was 0.3, occurring with a six-timestep delay. Thus, we conclude that simple cross-correlation was not particularly helpful in revealing the hidden common driver between the two observed variables in this case.

3.5. Evaluation and optimization of ASOM parameters

The ASOM training process includes several stochastic components; therefore, the results may vary when training is repeated on the same dataset. Moreover, the performance of the ASOM network depends on multiple parameters, including grid size, spatial and temporal learning parameters, the number of iterations, and the noise level in the data.

To assess the consistency of the results, we conducted experiments on data series from multiple simulated coupled logistic triads with fixed connections but varying logistic parameters and initial conditions (Eq. 4).

The temporal evolution of the learning process was evaluated by computing the absolute Pearson correlation between the estimated and ground truth hidden driver values on a 2000-step-long test dataset at every 10th training step in the outer loop. Here $N_1 = 40$, $N_2 = 20$ and $K = 10$ was used. Light green lines in Fig. 7(A) represent the evolution of

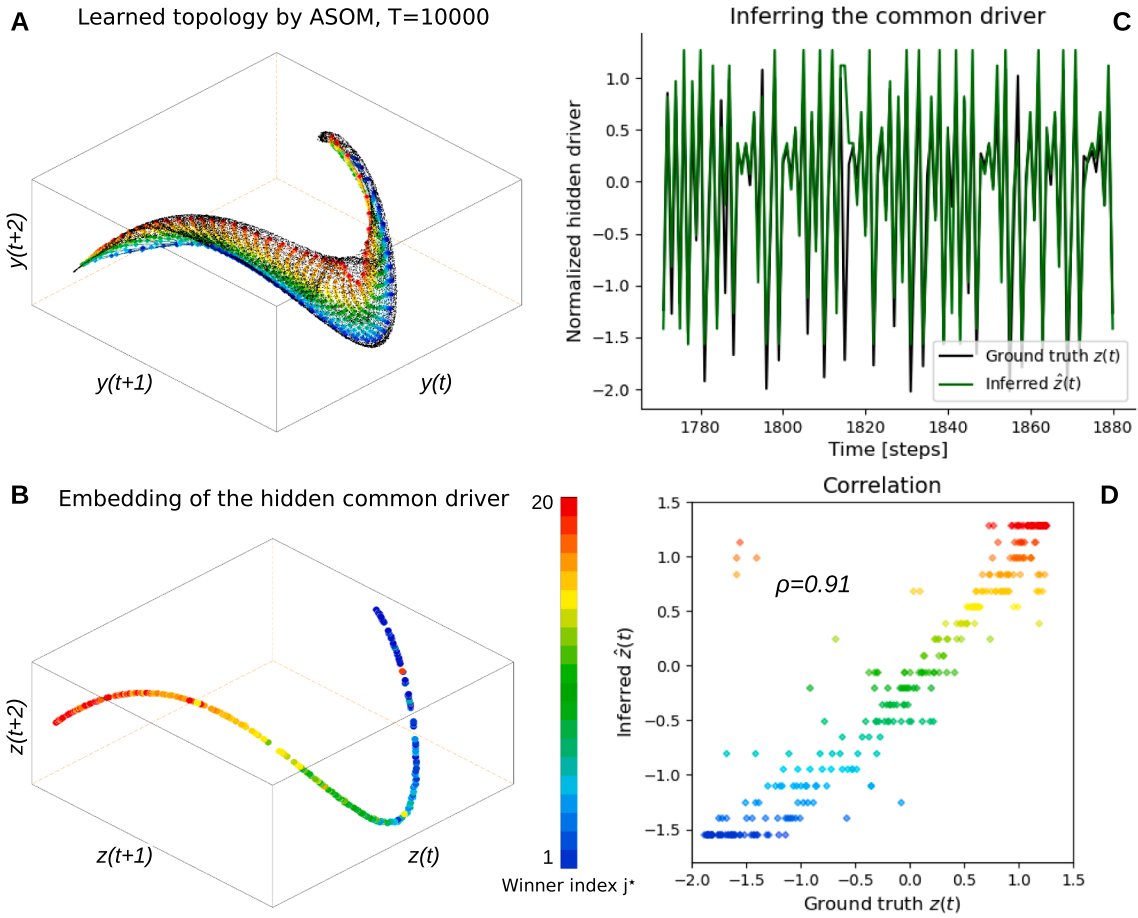


Figure 6. Readout of the hidden common driver. A: The trained 2D anisotropic self-organizing map (ASOM) after $N = 10000$ training steps, fits well to the Y manifold and divides it into fibers marked by different colors. The nodes of the grid represent the $C_{i,j}$ centers of the receptive fields of the neurons. The autonomous dynamics of system Y moves the system within the colored fibers of the ASOM, while fibers of different colors correspond to different states of the hidden common driver. B: The embedded state space of the ground truth hidden common driver, colored by the winner index j^* . The winner index j^* is marked with the same color in (B) as the corresponding fiber in (A). C: Comparison of the actual $z(t)$ (black) and the inferred hidden common driver $\hat{z}(t)$ (green). Both time series are normalized by medians and standard deviations. D: Correlation between the actual and reconstructed normalized hidden common driver values ($\rho = 0.91$). The color code corresponds to the winner index j^* . The striped pattern clearly shows the discretized nature of the inference.

the correlation coefficients across the 50 individual simulations, while the black line indicates their median.

The median correlation initially increases rapidly, followed by a slower but monotonic rise. However, not all individual training processes achieved high correlation by the end.

To assess the quality of the trained network, the fitting error E of the grid structure was calculated using the mean squared difference between the winner center points $C_{j^*,j^*}(t)$ and the vectors $Y(t)$ across the test dataset (Eq. 19). This evaluation can be performed without knowledge of the ground truth values of the hidden driver $z(t)$ and measures how well the center points are evenly distributed throughout the Y manifold. A lower fitting error E indicates a better alignment of the grid structure with the manifold.

Fig. 7(B) shows the temporal evolution of the fitting error: light green lines represent the evolution of the fitting error across individual training processes, while the black line indicates their median.

Following a brief initial increase at the start of training, the fitting error decreases monotonically. This suggests that the training process can be monitored by tracking the fitting error. However, the key question is whether the fitting error can reliably evaluate the quality of inference once training is complete.

Fig. 8(A) shows a clear negative correlation between the fitting error and the accuracy of hidden driver inference, measured by the absolute correlation achieved after 10,000 training steps using $N_1 = 40$, $N_2 = 20$

and $K = 10$ for the training. The analysis was conducted on 60 simulated logistic systems, with training repeated 11 times for each, resulting in a total of 660 training examples. While the correlation values exhibit some spread, small fitting error values consistently indicate high correlations. For example, restricting the results to cases where $E < 0.0002$ ensures correlation values above 0.8, demonstrating that the fitting error serves as a useful proxy for evaluating inference quality in the absence of ground truth data.

This property enables the comparison of two training processes on the same dataset, allowing selection of the higher correlation result by choosing the one with the lower fitting error. Repeating this process ten times leads to a continuous increase in the median correlation across $N = 60$ simulations (Fig. 8(B)), accompanied by a decrease in the median fitting error (Fig. 8(D)).

In this experiment, two different initial conditions for the $C_{i,j}(s = 0)$ values were compared. The dashed black lines in Fig. 8(B) represent the median, while the corresponding blue stripes indicate the quartiles and deciles of the training results when training started from randomly initialized centers. Additionally, the solid black lines and green percentile stripes show the results when training started from a regular grid in a “falling veil” configuration. While the difference was not significant, the correlation achieved with the regular initial configuration was consistently slightly higher at each step of the repetition; therefore, this setup was used for further experiments.

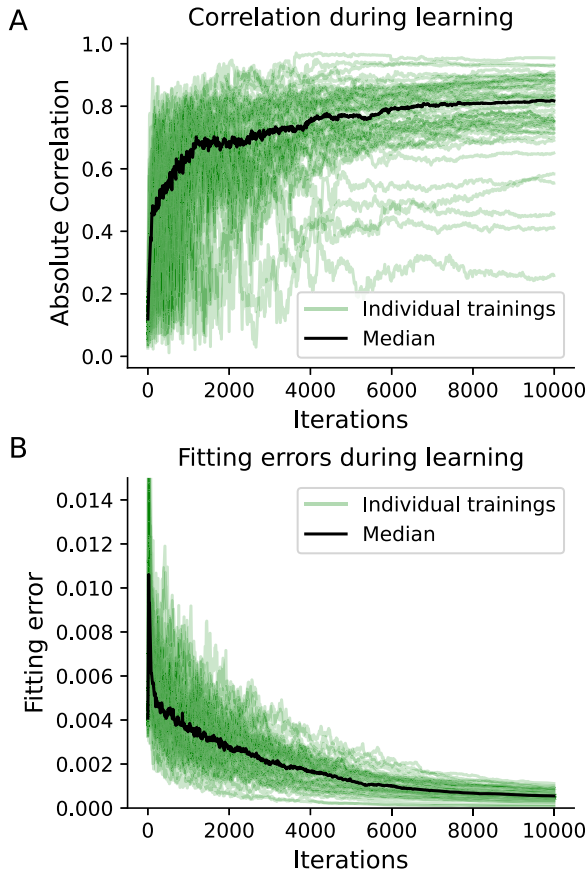


Figure 7. Evolution of absolute correlation and fitting error during learning. A: Light green lines represent the evolution of correlation across 50 individual learning curves, while the black line indicates their median. B: Light green lines represent the evolution of the fitting error across 50 individual learning curves, while the black line indicates their median.

Noise is unavoidable in real-world applications. To address this challenge, we tested the robustness of the ASOM procedure against noise (Fig. 8(C)). Coupled logistic map systems, similar to the previous test examples, were simulated with additive system noise. White noise samples were drawn from a normal distribution with varying standard deviations (SD). Median and percentile values are shown across $N = 30$ simulations for all noise levels. In this experiment, training remained stable up to an $SD = 0.02$ noise level. Given that the standard deviation of the logistic time series was 0.28, this noise level corresponds to approximately a signal-to-noise ratio of 7%.

The number of nodes along different dimensions is a critical parameter for our ASOM network. Fig. 8(E) shows the median correlation across $N = 30$ simulations as the grid sizes were varied from 10 to 60 nodes in steps of 10 along both dimensions. In almost all cases, the correlation monotonically increases with increasing N_1 , the number of nodes along the autonomous dynamics of the Y system. In contrast, increasing N_2 , the number of nodes along the hidden driver dimension, does not necessarily lead to higher correlation. Instead, the correlation exhibits a local maximum below $N_1/2$.

Since the training algorithm aims to distribute the nodes evenly along the manifold to be learned (weighted by the local sample density), it can be assumed that the shape of the grid should match the shape of the manifold for optimal results. Thus, elongated grids were necessary to learn the elongated manifolds in our case. This raises the question: how can the optimal shape of the network be determined in the absence of ground truth?

Higher node counts result in higher grid resolution and naturally lead to lower fitting errors. Therefore, the fitting error should be normalized

Table 1

Performance comparison of various methods. All methods were tested on 50-50 datasets of simulated coupled logistic and tent maps with two different lengths ($T = 5000$ and $T = 16000$). Performance was evaluated using the median absolute value of the Pearson correlation coefficient $|\rho|$ between the inferred and ground truth hidden driver data series, while variance was measured by the median absolute deviation of $|\rho|$. ASOM consistently achieved the highest median values across all four cases (highlighted in bold).

	Logistic map		Tent map	
	$T = 16000$	$T = 5000$	$T = 16000$	$T = 5000$
ASOM	0.821 ± 0.036	0.792 ± 0.041	0.854 ± 0.068	0.736 ± 0.115
DCCA	0.804 ± 0.060	0.735 ± 0.044	0.400 ± 0.144	0.428 ± 0.125
CCA	0.732 ± 0.037	0.734 ± 0.038	0.444 ± 0.124	0.441 ± 0.125
ShRec	0.725 ± 0.052	0.674 ± 0.059	0.425 ± 0.271	0.402 ± 0.228
PCA	0.713 ± 0.052	0.716 ± 0.050	0.374 ± 0.206	0.399 ± 0.199
KPCA	0.710 ± 0.051	0.715 ± 0.048	0.403 ± 0.184	0.410 ± 0.177
DCA	0.705 ± 0.037	0.704 ± 0.041	0.489 ± 0.089	0.494 ± 0.088
ICA	0.689 ± 0.038	0.695 ± 0.041	0.525 ± 0.072	0.525 ± 0.064
SOM	0.372 ± 0.110	0.423 ± 0.109	0.202 ± 0.085	0.240 ± 0.061
SFA	0.070 ± 0.032	0.143 ± 0.043	0.082 ± 0.046	0.040 ± 0.018
Random	0.035 ± 0.002	0.049 ± 0.004	0.144 ± 0.027	0.049 ± 0.003

to account solely for the quality of the fit (Eq. 20). This normalization allows for meaningful comparison of fitting errors across different grid sizes. Fig. 8(F) presents the median normalized fitting error values corresponding to the correlations shown in Fig. 8(E). A comparison of these graphs reveals that selecting N_2 to minimize the normalized fitting error for any given N_1 produces a grid shape close to the optimal and yields correlations near the maximum.

Finally, the inference performance of our ASOM method was compared to various general machine learning algorithms. While some of these, such as PCA, KPCA, ICA, DCA, and CCA, are commonly used to identify common variables among different observations, others, including SFA, DCCA, and ShRec, are specifically designed to reveal hidden common drivers based on observed time series. Among these alternative methods, PCA, ICA, DCA, and CCA employ linear models, KPCA extends the original linear algorithm by kernels to implement nonlinear dependencies, and SFA uses linear combinations of nonlinear functions. In contrast, traditional SOM, DCCA, and ShRec are fully nonlinear methods.

For these tests, two different dynamical systems were applied: a coupled triad of logistic maps (Eq. 4, similar to those used in previous examples) and a triad of coupled tent maps (Eq. 7). In the case of logistic maps, the connection strengths were fixed, while the parameters of the three maps and the initial conditions were randomized. However, for the tent maps, both the connection strengths and the mapping parameters were chosen randomly. A total of 50-50 data series were generated for both dynamical systems, and tests were conducted using training data of 5000 and 16000 steps in length. The ASOM network of size $N_1 = 40$, $N_2 = 20$ with $K = 10$ neighbourhood parameter was used.

The performance of each method was evaluated in terms of the median and median absolute error of the absolute correlation between the reconstructed and ground truth $z(t)$ values. These results are presented in Fig. 9 and summarized in Table 1.

Most methods achieved higher correlation values and lower variances on logistic map data. The only exception was our ASOM method, which resulted in higher correlation on the longer tent map dataset than on the longer logistic data. The variance in connection strength within the tent map system is reflected in the significantly higher variance observed across all methods.

Only a few methods benefited from the longer dataset, producing notably higher precision: our ASOM method on both time series, as well as DCCA and ShRec on the logistic maps. This aligns with the expectation that these more complex methods require and can effectively utilize longer training sessions.

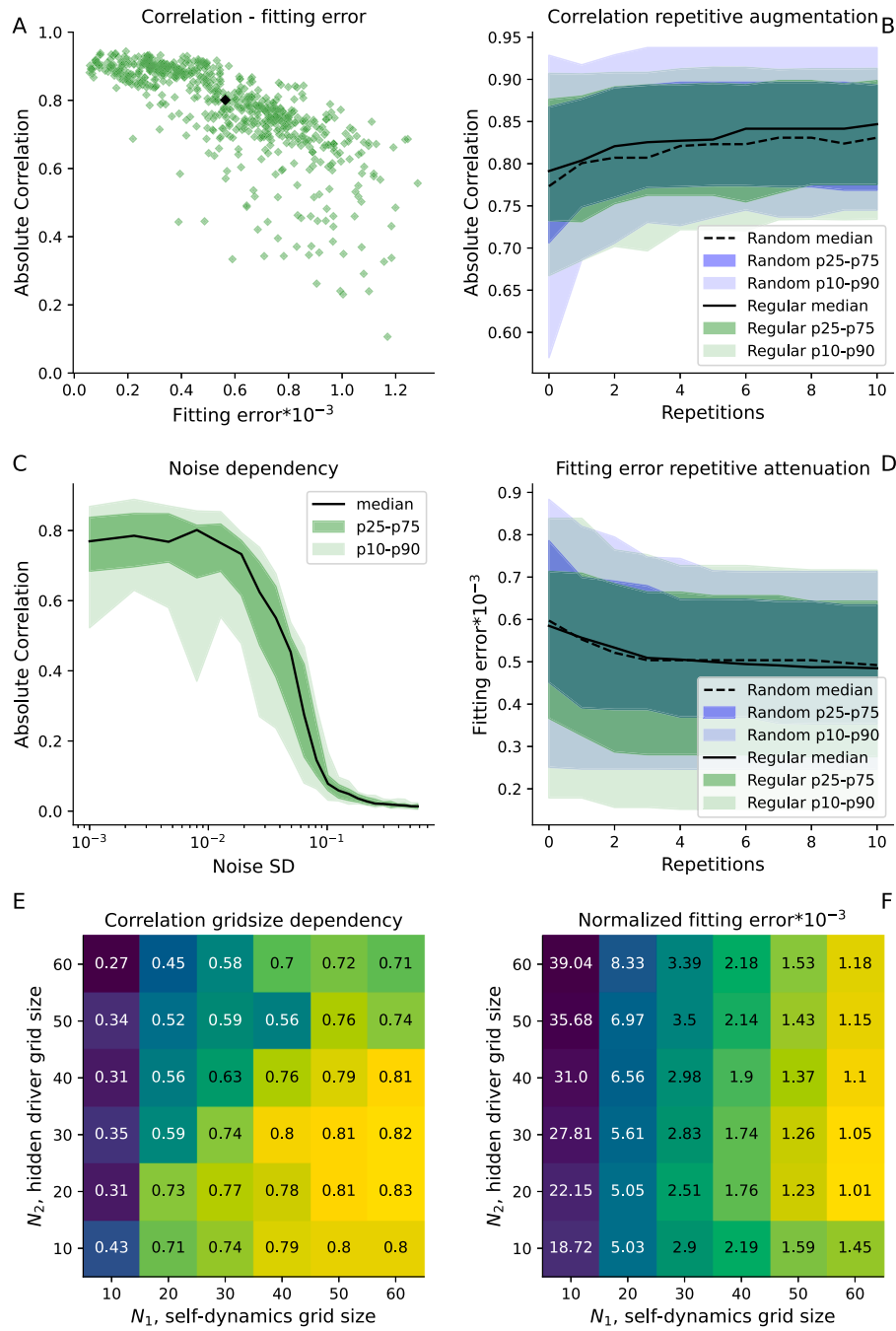


Figure 8. *Correlation and fitting error parameter dependence.* A: Anti-correlation between ASOM fitting error and hidden common driver inference, evaluated using the Pearson correlation coefficient. The black diamond represents the median. B: Improvement in correlation through repetitive fitting. The dashed line, along with darker and lighter blue stripes, represents the median, the 25–75 % percentile range, and the 10–90 % percentile range, respectively, when fitting started from randomly initialized grid center distributions. The solid line, along with darker and lighter green stripes, represents the same statistics when fitting started from regularly initialized grid center distributions. In each repetition, the result with the lower fitting error was selected. C: Correlation dependence on noise. The learning is stable up to noise level 0.02SD, that corresponds to 7 % signal-to-noise ratio. D: Monotonic decrease in fitting error during repetitive fitting. E: Dependence of correlation on grid size. F: Dependence of normalized fitting error on grid size. Note the inverse coloring in (E) and (F): lighter colors indicate higher correlation in (E) but lower error in (F).

The ranking of methods, in terms of correlation, differed between the two dataset families, with smaller variations observed between the different training lengths. However, for both types of dynamical systems and both dataset lengths tested, our ASOM method consistently achieved the highest median correlation values. In some cases, particularly for the tent maps, ASOM demonstrated significantly greater precision than any other method, achieving more than a 60 % higher correlation than the second-best method, the ICA.

The running time and performance, measured in terms of correlation, were evaluated using simulated coupled logistic map systems across different data lengths (Fig. 10). The results indicate that our ASOM implementation with $N = 20000$ iterations was the slowest among the evaluated algorithms. However, ASOM achieved the highest correlation with the hidden common driver time series for all data lengths greater than $T = 200$. Moreover, the running time of the ASOM algorithm scaled slowly with the increasing data length.

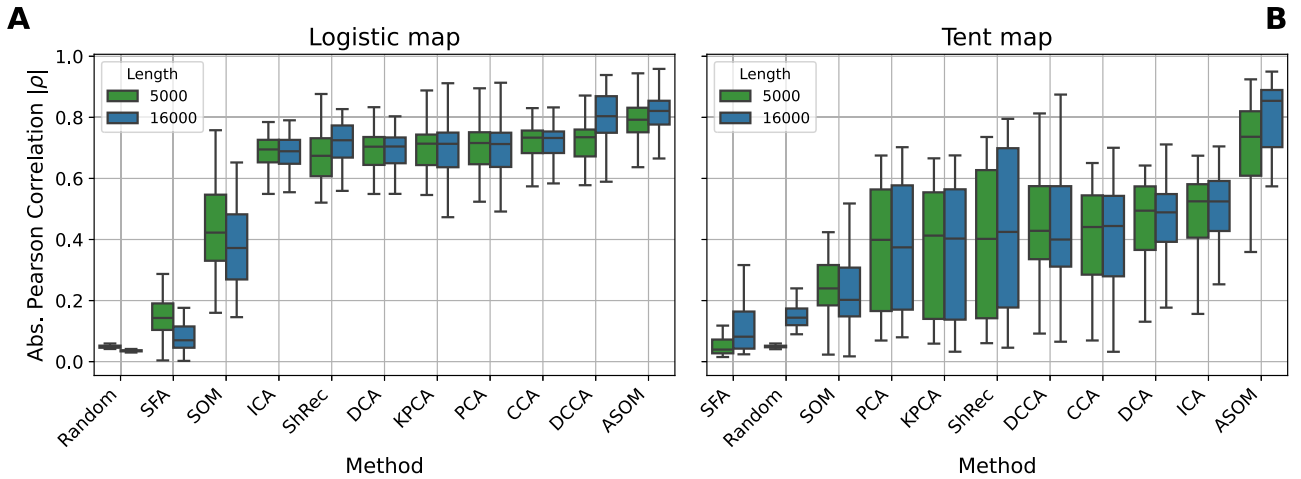


Figure 9. Inference performance comparisons. The inference performances are measured in terms of absolute correlation with the ground truth hidden common driver time series for 50-50 simulated systems of two different types: A: Logistic maps and B: Tent maps, with two different data lengths—green: 5000 steps and blue: 16000 steps. The medians are shown by solid black lines, the quartiles by green and blue boxes, and the 1.5 interquartile range by the whiskers. The methods compared are: Random baseline, Slow Feature Analysis (SFA), traditional self-organizing map (SOM), Independent Component Analysis (ICA), Dynamical Component Analysis (DCA), Principal Component Analysis (PCA), Kernel Principal Component Analysis (KPCA), Shared dynamics Reconstruction (ShRec) (Gilpin, 2023), Canonical Correlation Analysis (CCA), Deep Canonical Correlation Analysis (DCCA), and Anisotropic Self-Organizing Map (ASOM). Our ASOM method demonstrated superior precision in hidden common driver reconstruction for both dynamical systems and data series lengths compared to all other methods examined.

4. Discussion

In this paper, we have introduced a novel approach for inferring hidden common driver dynamics solely from time series observations of the driven systems. Our method builds upon the theoretical foundations of Takens' and Stark's theorems, similar to the works of Sauer (2010, 2004) and Gilpin (2025). Unlike these methods, we first estimated the intrinsic dimensions of the observed driven dynamical systems, then inferred the common driver's dimension by computing the mutual dimension of the observed systems. Based on these estimates, we constructed a neural network model using a self-organizing map and trained it with a new anisotropic variant of Kohonen's learning algorithm.

The effectiveness of our method was demonstrated on coupled triads of two types of discrete-time chaotic dynamical systems: coupled logistic and coupled tent maps. In all cases, two chaotic oscillators were driven by a third, similar one. We showed that, in a well-trained network, the second index of the winning node correlated well with the hidden common driver. After multiple simulations, we concluded that our ASOM method achieved more precise inference of the hidden common driver than several alternative methods.

Dimensionality plays a central role in topological comparisons: bidirectional topographical mapping can only exist between manifolds of equal dimensions and the different dimension of driver and driven manifolds results in asymmetric (unidirectional) CCM. In such cases, CCM implements a non-injective but surjective function from the higher-dimensional consequence to the lower-dimensional cause.

As we have seen, applying CCM from the cause to the consequence maps a small neighborhood of a point in the cause manifold to a submanifold in the consequence manifold, rather than spreading across the entire manifold. This poses challenges in evaluating CCM results, as the center of these submanifolds is typically non-independent of the cause's value. Consequently, CCM correlation in the reverse direction will be significantly different from zero (though smaller than one), preventing the application of statistical tests for clear conclusions. Additionally, this effect complicates distinguishing true causal interactions from noisy observations in the reverse direction (Sauer & Sugihara, 2025).

Similar to Sauer (2010, 2004) and Gilpin (2025), our approach leverages the property that the hidden common driver partitions the driven system's manifold into submanifolds, i.e., equivalence classes corresponding to the states of the hidden driver. Sauer's method collects

points belonging to the same equivalence class and selects a representative. However, this process was carried out on systems, where the driven manifold is 1-dimensional only (Sauer, 2010, 2004). Although dividing a 1D manifold into equivalence classes is straightforward due to the proximity of class members, partitioning a 2D manifold into 1D submanifolds is more challenging. Our method addresses this by fitting a neural grid with predefined dimensions.

Our examples involved the case of a 1D driver and 2D driven systems, where a 2D ASOM grid was sufficient to capture both the driven system's dynamics and the 1D driver dynamics within it. However, higher-dimensional systems will introduce additional challenges, requiring further investigation.

If additional assumptions can be made about the nature of the hidden common driver, alternative reconstruction methods become available. For instance, Slow Feature Analysis (SFA) can infer the hidden common driver under the assumption that its dynamics evolve more slowly than those of the driven systems (Wiskott, 1998; Wiskott & Sejnowski, 2002). In contrast, our approach does not rely on such assumptions; in our demonstration, both the hidden driver and the driven systems exhibited fast, chaotic dynamics with similar characteristic frequencies.

Similarly, statistical methods such as PCA, ICA, CCA, and DCA approximate a hidden common driver under the assumption that the observed time series are linear combinations of the hidden variables. Again, our approach does not impose such constraints; in our demonstration, linear correlations between the observed and hidden systems were weak and time-shifted.

The key distinction between a traditional SOM and our ASOM lies in the ability of the latter to utilize the relationship between the two manifolds of the observed dynamical systems, rather than fitting to a single one. This relationship defines the transversal manifold structure, separating dimensions corresponding to the hidden common driver from those of autonomous dynamics, giving ASOM a significant advantage. Our simulations demonstrated that the even node distribution achieved by a traditional SOM does not align with the dynamical structure of the embedded manifolds.

While our method achieved the highest precision in inferring the hidden common driver, it was also the slowest, at least in its current implementation. This trade-off between performance and running time should guide the choice of inference method for a given application. If the primary goal is to achieve maximal accuracy and

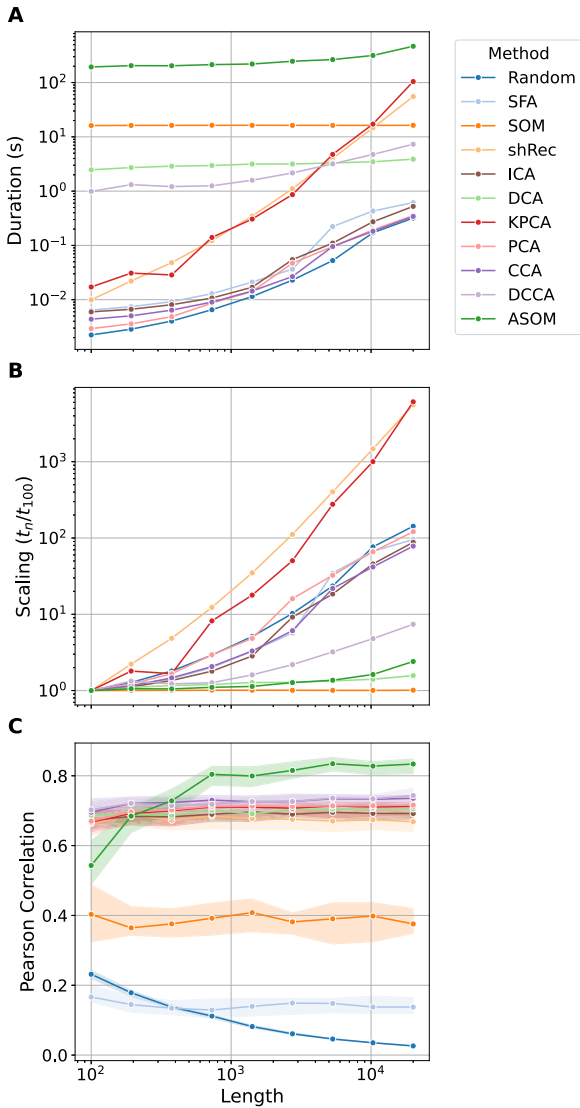


Figure 10. Running time and performance comparison evaluated on simulated logistic map systems. A: Running time of the different methods as a function of data length. B: Scaling of the running time, normalized to the shortest dataset ($T = 100$). C: Performance measured by correlation with the ground truth hidden common driver time series. All graphs display median values and 95 % confidence intervals of the median; however, in panels A and B, the confidence interval is nearly imperceptible. Our ASOM implementation with $N = 20000$ iterations was the slowest among the evaluated methods in terms of time, but achieved the highest performance for data lengths greater than $T = 200$ time steps. Furthermore, the running time of ASOM scales slowly with the increasing data length.

there are no strict time constraints, our method provides a clear advantage.

It is reasonable to assume that even non-linearly coupled hidden common driver time series may emerge in the hidden layers of deep autoencoders trained on the observed time series. However, identifying and extracting the relevant features from the large number of neurons in hidden layers remains a significant challenge.

To address this issue, Benkő and Somogyvári (2021) proposed a feed-forward deep neural network called the Mapper-Coach architecture. This approach uses error backpropagation to train the network in predicting the future of one time series based on both observed dynamics. The estimation of the hidden common driver naturally emerges in the information bottleneck between the two modules. Notably, prior

estimation of the dimensionality of the hidden common driver is crucial in this framework, as defining the bottleneck width appropriately is essential for effective reconstruction.

5. Conclusions

Our ASOM algorithm successfully leveraged the manifold structure, whose existence is guaranteed by general theorems of differential topology. Proper learning of this structure enabled more precise inference of the hidden common driver compared to many widely used or recently developed algorithms.

While properly determining all ASOM parameters is challenging in the absence of ground truth data, we demonstrated that the quality of training results can be evaluated by calculating the fitting error of the ASOM grid. This fitting error serves as a reliable metric for comparing inference quality across different parameter setups, even without access to ground truth data.

Our current work is restricted to 1+1 dimensional dynamics. While the theoretical foundation suggests that the decomposition of driven attractors into transversal submanifolds should extend to higher-dimensional systems, future research is needed to determine whether ASOM can effectively identify these structures in higher-dimensional settings.

CRediT authorship contribution statement

Zsigmond Benkő: Visualization, Software, Investigation, Data curation; **Marcell Stippinger:** Software; **Attila Bencze:** Validation, Conceptualization; **Fülöp Bazzó:** Writing – review & editing, Formal analysis; **András Telcs:** Investigation, Formal analysis, Conceptualization; **Zoltán Somogyvári:** Writing – original draft, Validation, Supervision, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper. dr. Zoltán Somogyvári is an employee of the Neunos LTD. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

This research was supported by the Hungarian National Research, Development, and Innovation Office NKFIH, under grant numbers K113147 and K135837, the Hungarian Research Network HUN-REN under grant numbers SA-114/2021 and TECH-2024-20, and by the Hungarian National Brain Research Program 2017-1.2.1-NKP-2017-00002.

Appendix A. Simulations and ASOM trainings

A.1. Simulations

Table A.2 summarizes the parameters of the dynamical systems used in various simulations in our study. For individual simulations, the values of the logistic parameter r were chosen to induce chaotic behavior, while for systematic performance evaluations, r was sampled uniformly from the intervals specified in Table A.2.

Our tilted tent map produced chaotic behavior for almost all values of α ; however, to generate diverse dynamics, the parameters of α were also randomly chosen. The initial conditions for $x(t)$, $y(t)$, and $z(t)$ were uniformly sampled from the interval $[0, 1]$ in all cases.

Table A.2
Parameters of simulations.

Figures	System	Structure	Parameter (r)	Coupling coeff (β_x, β_y)	Noise ($SD(\eta)$)
Fig. 2 A-D	Logistic	Circular	3.86	0.5,0.6	0
Fig. 2 E-H	Logistic	Unidirectional	3.86	0.5	0
Fig. 3	Logistic	Hidden driver	3.8	0.3,0.4	0
Fig. 4	Logistic	Hidden driver	3.8	0.3,0.4	0
Fig. 5	Logistic	Hidden driver	3.8	0.3,0.4	0.001
Fig. 6	Logistic	Hidden driver	3.8	0.3,0.4	0.001
Fig. 7	Logistic	Hidden driver	3.8	0.3,0.4	0
Fig. 8	Logistic	Hidden driver	3.8-4	0.3,0.4	0
Fig. 8(C)	Logistic	Hidden driver	3.8-4	0.3,0.4	0.001-1
Fig. 9	Logistic	Hidden driver	3.8-4	0.3,0.4	0
Fig. 9	Tent map	Hidden driver	0.1-0.5 (α)	0.1-0.5	0

Table A.3

Parameters of the ASOM training. For random initial conditions, the vectors $C_{i,j}(s=0)$ were uniformly distributed within the unit cube. For regular initial conditions, the vectors $C_{i,j}(s=0)$ were placed on a regular grid over a unit square at a height of 1 as the third coordinate, forming a “falling veil” during the initial phase of learning.

Figures	Grid size $N_1 \times N_2$	Neighbors K	Data length T	Initial condition $C_{i,j}(s=0)$	Iterations N	Repetitions
Fig. 5	40×20	20	20000	Random	1000	1
Fig. 6	40×20	20	20000	Random	10,000	1
Fig. 7	40×20	10	20000	Regular	10,000	50
Fig. 8(A)	40×20	10	20000	Regular	10,000	11×60
Fig. 8(B), (D)	40×20	10	20000	Random, Regular	10,000	11×60
Fig. 8(C)	40×20	10	20000	Regular	10,000	11×30
Fig. 8(D)	40×20	10	20000	Regular	10,000	11×60
Fig. 8(E), (F)	$10 - 60 \times 10 - 60$	10	20000	Regular	10,000	36×60
Fig. 9(A)	40×20	10	5000, 16000	Regular	500,016,000	50
Fig. 9(B)	40×20	10	5000, 16000	Regular	500,016,000	50
Fig. 10	40×20	10	100–20000	Regular	20,000	50

The robustness of our method to noise was systematically tested by adding Gaussian white noise with varying standard deviations ($SD(\eta)$) to the simulated variables at each time step.

For systematic evaluation and comparison across methods, the simulations were repeated between 30 and 60 times, depending on the specific experiment.

A.2. ASOM trainings

Table A.3 summarizes the parameter settings applied for ASOM training, which were systematically varied across different experiments. The tested parameters included two types of initial conditions (Random and Regular), neighborhood sizes ($K = 10$ or 20), grid sizes ranging from 10 to 60 per dimension, the number of training iterations ($1,000$ to $20,000$), and data lengths.

To evaluate the performance of different methods, we conducted a series of comparisons by varying the data length between 100 and $20,000$ samples, thereby capturing performance trends across a wide range of sample sizes. For each data length, we measured both the correlation between the estimated and true values and the computational runtime to assess the accuracy and efficiency of each method. All computations were performed on an Intel(R) Xeon(R) CPU X5650 @2.67 GHz system running Ubuntu 20.04.6 LTS.

B. Alternative methods

Several previously proposed methods for extracting common components from multivariate time series are listed here. The performance and the efficiency of our ASOM method is compared to these methods.

B.1. Slow feature analysis (SFA)

Slow Feature Analysis (Wiskott, 1998; Wiskott & Sejnowski, 2002) was performed on the preprocessed signals using the `sksfa` (version 0.1.6) Python package.

We used the embedded signals from the observed variables purely as input features and set the number of components parameter to one. The same procedure was applied to the tent map dataset.

B.2. Traditional SOM

Kohonen’s self-organizing map (SOM) was implemented using the same parameters as in the ASOM. However, the training procedure was simpler: in each step, a randomly chosen $Y(t_s)$ element was presented to the network, the winner node was determined, and the node centers were updated. This step was repeated $N \cdot K$ times to reach the same level of training as in the ASOM, after which the node centers were frozen. Finally, the test set was presented, and the coordinate indices of the winner node were recorded.

Although the grid size was 40×20 , the same as for the ASOM, a key difference was that here we did not know a priori which dimension of the SOM grid corresponded to the hidden driver dynamics. To give the traditional SOM an advantage, we tested both dimensions and selected the one with the higher correlation to the ground truth for each dataset.

B.3. Principal component analysis (PCA)

We applied PCA using the `scikit-learn` package (Pedregosa et al., 2011). For the logistic map and tent map datasets, we passed the embedded input signals from the observed subsystems and projected them onto the first principal component. We then calculated the correlation between the projection and the hidden variable.

B.4. Kernel principal component analysis (KPCA)

The Kernel PCA (Schölkopf et al., 1997) was calculated using the `scikit-learn` package (Pedregosa et al., 2011). We used the default parameters of the class, applied the radial basis function kernel, and set the number of components to one for the calculations.

B.5. Independent component analysis (ICA)

ICA transforms a multivariate signal into components by minimizing the mutual information between them to achieve an effective data representation. We used the ICA implementation from the `scikit-learn` package (Pedregosa et al., 2011).

A key parameter of the method is the number of components onto which the original high-dimensional signal is projected. We set this parameter to 2 for the logistic map dataset and 3 for the tent map dataset. To provide an advantage for the ICA method, the component with the highest correlation was selected for evaluation in each dataset.

B.6. Dynamical Component Analysis (DCA)

Dynamical Component Analysis (DCA) is a linear dimensionality reduction technique that projects high-dimensional time series into a subspace with maximal predictive information, measured by mutual information (Clark et al., 2019). We used the official Python implementation from GitHub (Clark et al., 2021).

DCA has two main parameters: the time window and the number of components used in the procedure. We set the time window parameter to $T = 5$ and the number of components to one for both datasets.

B.7. Canonical Correlation Analysis (CCA)

We applied Canonical Correlation Analysis (CCA) to linearly project the observed subsystems onto subspaces where the correlation between their representations is maximized. We used the `scikit-learn` implementation of CCA (Pedregosa et al., 2011).

For the logistic map and tent map datasets, we embedded the time series into three dimensions and set the number of components to one. We then computed a projected value for each of the observed subsystems, averaged the two, and used this value as an estimate for the hidden variable.

B.8. Deep Canonical Correlation Analysis (DCCA)

We applied Deep Canonical Correlation Analysis (DCCA) to map the observed subsystems onto 1D subspaces, where the correlation between their representations is maximized (Andrew et al., 2013). We used the DCCA implementation from the `mvlearn` Python package (Perry et al., 2021). The same data processing procedure was applied as in the case of linear CCA.

For the neural network architecture, we used multilayer feedforward networks with two hidden layers, each containing $m = 20$ hidden units.

B.9. Shared Dynamics Reconstruction Algorithm (ShRec)

We applied the Shared Dynamics Reconstruction Algorithm (ShRec) to recover the hidden driver behind the observed subsystems (Gilpin, 2025). We utilized the official Python implementation available on GitHub, given in the preprint (Gilpin, 2023). The embedding parameter was set to 3, while all other parameters were left at their default values. We then compared the reconstructed hidden variable to the predicted shared dynamics.

B.10. Random baseline (random)

To establish a random baseline for each dataset, we generated a normally distributed random signal with zero mean and a standard deviation of 1, then computed its correlation with the ground truth hidden driver time series.

References

- Andrew, G., Arora, R., Bilmes, J., & Livescu, K. (2013). Deep canonical correlation analysis. In *Proceedings of the 30th international conference on machine learning*, PMLR 28(3) (pp. 1247–1255).
- Assaad, C. K., Devijver, E., & Gaussier, E. (2022). Survey and evaluation of causal discovery methods for time series. *Journal of Artificial Intelligence Research*, 73. <https://doi.org/10.1613/jair.1.13428>
- Banyaga, A. (2004). Lectures on morse homology. *Texts in the mathematical sciences*, (pp. 130). Springer.
- Benkő, Z., Moldován, K., Szádeczky-Kardoss, K., Zálányi, L., Borbély, S., Világi, I., & Somogyvári, Z. (2019). Causal relationship between local field potential and intrinsic optical signal in epileptiform activity in vitro. *Scientific Reports*, 9(1). <https://doi.org/10.1038/s41598-019-41554-x>
- Benkő, Z., & Somogyvári, Z. (2021). Reconstructing shared dynamics with a deep neural network. (pp. 1–7). <https://doi.org/10.48550/arXiv.2105.02322>
- Benkő, Z., Stippinger, M., Rehus, R., Bencze, A., Fabó, D., Hajnal, B., Erőss, L., Telcs, A., & Somogyvári, Z. (2020). Manifold-adaptive dimension estimation revisited. *PeerJ Computer Science* 8:e790. <https://doi.org/10.7717/peerj-cs.790>
- Benkő, Z., Varga, B., Stippinger, M., & Somogyvári, Z. (2025). Detecting causality in the frequency domain with cross-mapping coherence. *Physica D: Nonlinear Phenomena*, 477, 134708. <https://doi.org/10.1016/j.physd.2025.134708>
- Benkő, Z., Zlatniczki, Á., Stippinger, M., Fabó, D., Sólyom, A., Erőss, L., Telcs, A., & Somogyvári, Z. (2018). Complete inference of causal relations between dynamical systems. (pp. 1–21). [arXiv:1808.10806](https://arxiv.org/abs/1808.10806).
- Benkő, Z., Zlatniczki, Á., Stippinger, M., Fabó, D., Sólyom, A., Erőss, L., Telcs, A., & Somogyvári, Z. (2024). Bayesian inference of causal relations between dynamical systems. *Chaos, Solitons & Fractals*, 185, 115142. <https://doi.org/10.1016/j.chaos.2024.115142>
- Chu, T., & Glymour, C. (2008). Search for additive nonlinear time series causal models. *Journal of Machine Learning Research*, 9(32), 967–991. <http://jmlr.org/papers/v9/chu08a.html>.
- Clark, D., Livezey, J., & Bouchard, K. (2021). Dynamical components analysis. <https://github.com/BouchardLab/DynamicalComponentsAnalysis>.
- Clark, D. G., Livezey, J. A., & Bouchard, K. E. (2019). Unsupervised discovery of temporal structure in noisy data with dynamical components analysis. In *Proceedings of the 33rd international conference on neural information processing systems* (pp. 1–12). Red Hook, NY, USA: Curran Associates Inc.
- Farahmand, A.-M., Szepesvári, C., & Audibert, J.-Y. (2007). Manifold-adaptive dimension estimation. In *Proceedings of the 24th international conference on machine learning - ICML '07* (pp. 265–272). New York, New York, USA: ACM Press.
- Gilpin, W. (2023). Recurrences reveal shared causal drivers of complex time series. (pp. 1–11). [arXiv:2301.13516](https://arxiv.org/abs/2301.13516).
- Gilpin, W. (2025). Recurrences reveal shared causal drivers of complex time series. *Physical Review X*, 15, 011005. <https://doi.org/10.1103/PhysRevX.15.011005>
- Glymour, C., Zhang, K., & Spirtes, P. (2019). Review of causal discovery methods based on graphical models. *Frontiers in Genetics*, 10, 524. <https://doi.org/10.3389/fgene.2019.00524>
- Gosztolai, A., Peach, R. L., Arnaudon, A., Barahona, M., & Vanderghelynst, P. (2024). Interpretable statistical representations of neural population dynamics and geometry. <https://doi.org/10.48550/arXiv.2304.03376>
- Hirata, Y., & Aihara, K. (2010). Identifying hidden common causes from bivariate time series: A method using recurrence plots. *Physical Review E*, 81(1), 016203. <https://doi.org/10.1103/PhysRevE.81.016203>
- Kohonen, T. (1982). Self-organized formation of topologically correct feature maps. *Biological Cybernetics*, 43(1), 59–69. <https://doi.org/10.1007/bf00337288>
- Kohonen, T. (1990). The self-organizing map. *Proceedings of the IEEE*, 78(9), 1464–1480. <https://doi.org/10.1109/5.58325>
- Pearl, J. (2000). Causality: Models, reasoning, and inference. Cambridge University Press.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Perry, R., Mischler, G., Guo, R., Lee, T., Chang, A., Koul, A., Franz, C., Richard, H., Carmichael, I., Ablin, P., Gramfort, A., & Vogelstein, J. T. (2021). Mvlearn: Multiview machine learning in python. *Journal of Machine Learning Research*, 22(109), 1–7.
- Runge, J. (2018). Causal network reconstruction from time series: From theoretical assumptions to practical estimation. *Chaos: An Interdisciplinary Journal of Nonlinear Science*, 28(7), 075310. <https://doi.org/10.1063/1.5025050>
- Sard, A. (1942). The measure of the critical values of differentiable maps. *Bulletin of the American Mathematical Society*, 48(12), 883–890. <https://doi.org/10.1090/S0002-9904-1942-07811-6>
- Sauer, T. (2010). Detection of periodic driving in nonautonomous difference equations. In *Lecture notes in mathematics* (pp. 301–309). (Vol. 2002). <https://doi.org/10.2969/aspm/05310301>
- Sauer, T., & Sugihara, G. (2025). Robust methods to detect coupling among nonlinear dynamic time series. *Physical Review E*, 111, 064208. <https://doi.org/10.1103/PhysRevE.111.064208>
- Sauer, T. D. (2004). Reconstruction of shared nonlinear dynamics in a network. *Physical Review Letters*, 93(19), 1–4. <https://doi.org/10.1103/PhysRevLett.93.198701>
- Schölkopf, B., Smola, A., & Müller, K.-R. (1997). Kernel principal component analysis. In W. Gerstner, A. Germond, M. Hasler, & J.-D. Nicoud (Eds.), *Artificial neural networks — ICANN'97* (pp. 583–588). Berlin, Heidelberg: Springer Berlin Heidelberg.
- Schreiber, T. (2000). Measuring information transfer, 85(2), 461–464. American Physical Society. <https://doi.org/10.1103/PhysRevLett.85.461>

- Stark, J. (1999). Delay embeddings for forced systems. I. Deterministic forcing. *Journal of Nonlinear Science*, 9, 255–332. <https://doi.org/10.1007/s003329900072>
- Stark, J., Broomhead, D. S., Davies, M. E., & Huke, J. (2003). Delay embeddings for forced systems. II. stochastic forcing. *Journal of Nonlinear Science*, 13(6), 519–577. <https://doi.org/10.1007/s00332-003-0534-4>
- Sugihara, G., May, R., Ye, H., Hsieh, C.-H., Deyle, E., Fogarty, M., & Munch, S., et al. (2012). Detecting causality in complex ecosystems. *Science (New York, N.Y.)*, 338(6106), 496–500. <https://doi.org/10.1126/science.1227079>
- Sugiyama, M., & Borgwardt, K. M. (2013). Measuring statistical dependence via the mutual information dimension. *IJCAI international joint conference on artificial intelligence*, (pp. 1692–1698).
- Takens, F. (1981). Detecting strange attractors in turbulence. *Dynamical Systems and Turbulence, Warwick 1980*, 898, 366–381. <https://doi.org/10.1007/BFb0091924>
- Telcs, A., Kurbucz, M. T., & Jakovác, A. (2025). Unified causality analysis based on the degrees of freedom. *Physical Review E*, 112, 014204. <https://doi.org/10.1103/PhysRevE.112.014204>
- Wiskott, L. (1998). Learning invariance manifolds. In *Proc. 8th intl. conf. on artificial neural networks (ICANN'98)* (pp. 555–560).
- Wiskott, L., & Sejnowski, T. J. (2002). Slow feature analysis: Unsupervised learning of invariances. *Neural Computation*, 14(4), 715–770. <https://doi.org/10.1162/089976602317318938>
- Wójcik, D., Nowak, A., & Kuś, M. (2001). Dimension of interaction dynamics. *Physical Review E*, 63, 036221. <https://doi.org/10.1103/PhysRevE.63.036221>
- Ye, H., Deyle, E. R., Gilarranz, L. J., & Sugihara, G. (2015). Distinguishing time-delayed causal interactions using convergent cross mapping. *Scientific Reports*, 5, 14750.